24-hour join acceptance run (#4559)

Run 20261013T045549Z — dagbft-integration, 5 tps, one late follower at a time, no node disturbed

Soak harness, test/docker/soak; report compiled 2026-10-14

Oct 12 23:55 CT – Oct 13 23:58 CT (24.01 h)

Verdict

The bar is met. Every one of the 82 followers bootstrapped against the running network reached ACTIVE on both of its partitions within two minutes (bar: one hour), held ACTIVE for the whole 15-minute measurement window, and neither the join time nor the lag behind the validators degraded over the 24 hours beyond what the growing load-generator universe accounts for. The validators ran the full day without an out-of-memory kill, a GC thrash, a wedge, a delivery stall or a heal.

criterion (#4559 / join-acceptance)	result
follower syncs within an hour	82 of 82; added → ACTIVE 21–102 s, median 61 s
every section loaded within an hour	82 of 82; ≤ 1.6 min
holds ACTIVE under load	0 of 7,380 polls not ACTIVE; worst behind 1 block (BVN), 2 blocks (Directory)
no degradation over 24 h	join time grew 41 s (first ten) → 77 s (last ten), tracking the universe 131 → 20,305 accounts; lag flat at ≤ 1 block
network undisturbed	0 heals, 0 wedge captures, 0 delivery-stall captures, 0 reconcile pulls, 0 stalled channels at the end
memory	no OOM, no GC thrash; both GOMEMLIMIT arms stayed under their container limits

What this run does **not** show: 10 tps (the issue text named the 10 tps conf; this ran at 5), any disturbance (nothing was paused, restarted or killed), and history older than 24 hours — a follower joining a network with weeks of blocks was not measured. What the day's slopes imply for a network running for years is the section of that name.

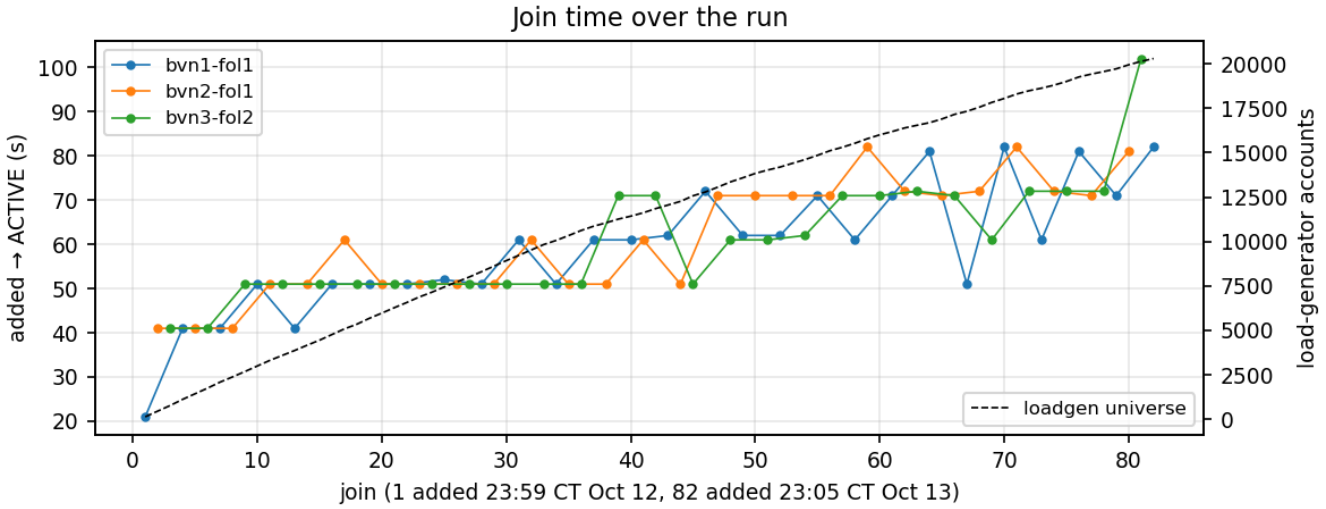
Setup

field	value
code	`dagbft-integration` at `6daa4fd48` (branch `test-4530-4542`), executor v2-kourou
topology	3 BVNs × 4 validators, Directory on all 12; one follower (`bvn3-fol1`) launched with the network; three late followers (`bvn1-fol1`, `bvn2-fol1`, `bvn3-fol2`) started one at a time by the walk
storage	BlockchainDB (`docker-network.yml`); 1 s blocks

field	value
load	5 tps mixed transactions, 100 bootstrap accounts; 454,781 generated, 0 rejected, 5.26 tps achieved
walk	`24h-5tps-followers.conf`: databases cleared, follower started, up to 1800 s to ACTIVE on every partition, held 900 s under measurement, removed; round-robin bvn1 → bvn2 → bvn3; first add 2 min in
memory A/B (the compose)	val1/val2 of each BVN: GOGC 200, GOMEMLIMIT 2000 MiB, mem_limit 2348 MiB; val3/val4 and followers: GOMEMLIMIT 1300 MiB, mem_limit 2048 MiB; restart policy `no`
recorded	per node every 30 s: RSS, heap, next GC, GC count, CPU seconds, goroutines (`mem.csv`); block heights per node/partition (`monitor.csv`); follower lag (`follower.csv`); host disk and CPU every 10 s (`hostsample.csv`); hourly heap, 30 s CPU, goroutine and block profiles plus metrics from every node (23 sets, 757 MB)
host	thelio, shared NVMe `nvme0n1`; the weekly fstrim (Mondays ~00:20 CT) did not fall in the window

Joins

82 joins, 82 PASS. Each join is a fresh database: the follower pulls the state, matches the validators' root and BPT, is declared ACTIVE when it executes in agreement on both partitions, then fills the remaining sections in the background.



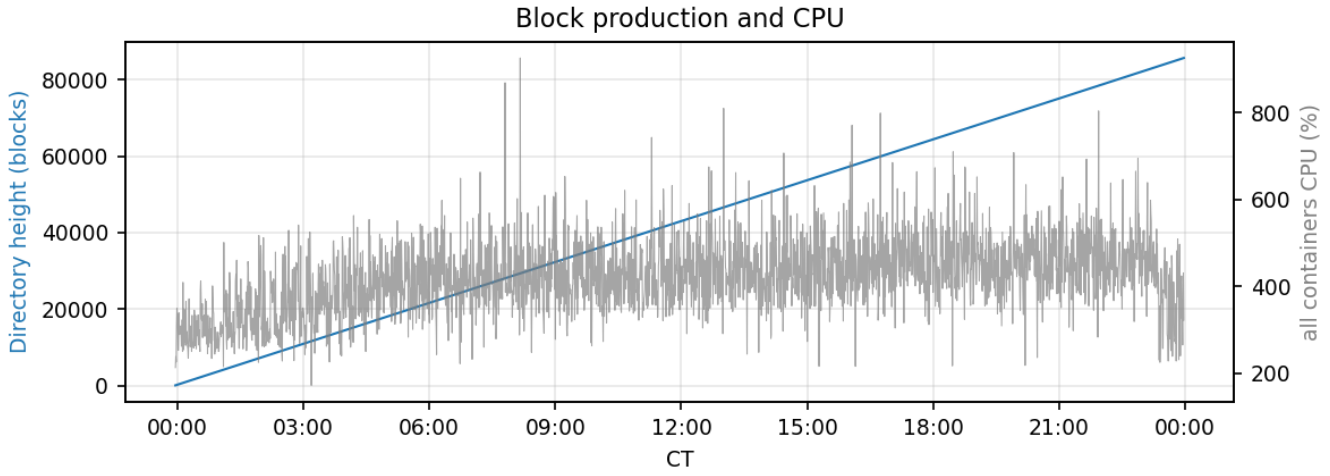
	bvn1-fol1	bvn2-fol1	bvn3-fol2
joins	28	27	27
added → ACTIVE, min / median / max (s)	21 / 61 / 82	41 / 61 / 82	41 / 61 / 102
worst behind during the hold, BVN / Directory (blocks)	1 / 1	1 / 2	1 / 1
polls not ACTIVE	0	0	0

The join time climbs about 10 s for every 2,500 accounts the load generator adds to its universe (131 at join 1, 20,305 at join 82): the state pull is proportional to the account set, not to the chain's height (85,674 Directory blocks at the end). The three BVNs are indistinguishable. The one 102 s join (81, bvn3-fol2 at 22:47 CT) is the largest; its fill completed at 1.6 min and its hold was clean.

"Filled" (every section loaded, from `joinphases.py`) ran 0.2 min at the start to 1.2–1.6 min at the end, always within a minute of ACTIVE. The full per-join table is in the appendix.

Removals were judged on the 30 s either side: block cadence unaffected on every partition, every time (`manifest.md`, per-event table).

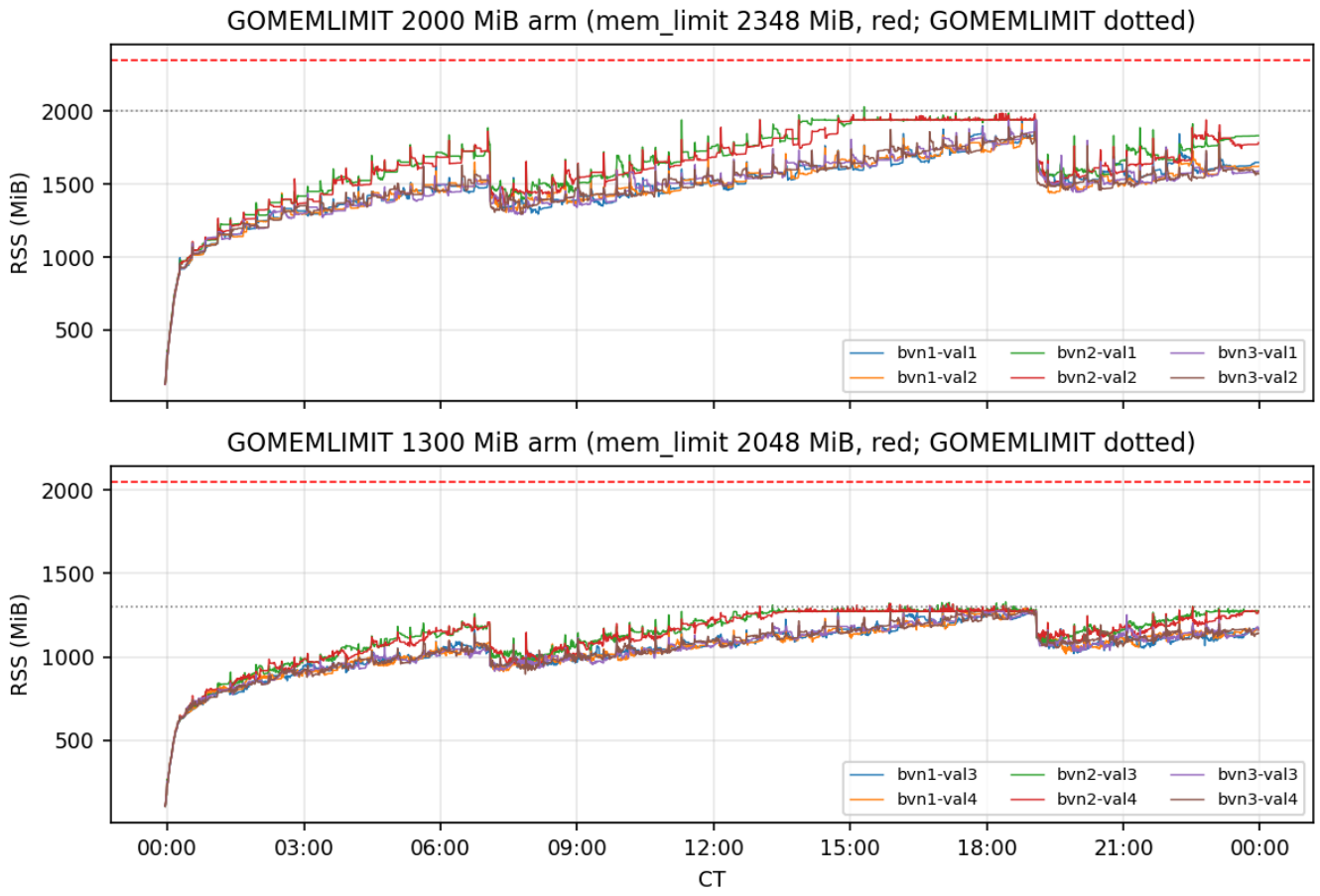
Validators over the day

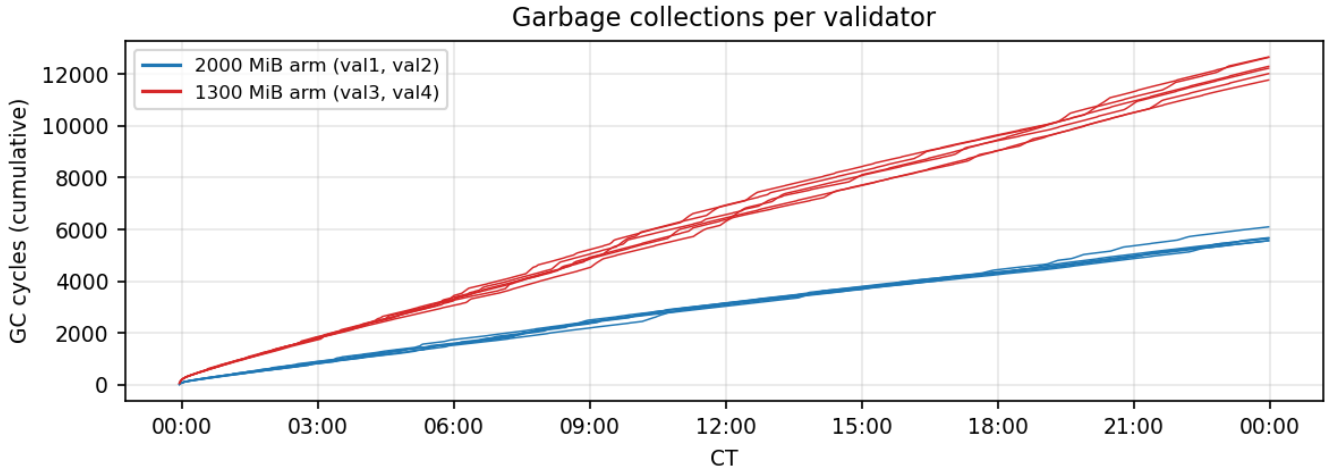


The Directory closed 85,674 blocks in 24.01 h (0.99 blocks/s against a 1 s interval). Whole-network container CPU (all 16–17 containers, `docker stats`) ran at a median 419 % of one core, peaking at 926 % during joins. Heals stayed at 0 from the first sample to the last.

Memory

Both arms of the compose A/B ran the whole day.





node	arm (GOMEM-LIMIT / mem_limit)	RSS hour 1 (median)	RSS hour 24 (median)	RSS peak	GC cycles	CPU s	goroutines peak / end
bvn1-val1	2000 / 2348	944	1624	1939	6,079	23,569	332 / 292
bvn1-val2	2000 / 2348	951	1613	1914	5,549	21,250	344 / 297
bvn1-val3	1300 / 2048	662	1137	1289	12,238	23,369	338 / 294

node	arm (GOMEM-LIMIT / mem_limit)	RSS hour 1 (median)	RSS hour 24 (median)	RSS peak	GC cycles	CPU s	goroutines peak / end
bvn1-val4	1300 / 2048	665	1146	1293	11,961	23,551	335 / 293
bvn2-val1	2000 / 2348	992	1827	2030	5,660	24,194	344 / 296
bvn2-val2	2000 / 2348	997	1773	1987	5,635	23,642	340 / 294
bvn2-val3	1300 / 2048	684	1274	1329	12,602	26,138	333 / 291
bvn2-val4	1300 / 2048	698	1249	1321	11,711	24,116	340 / 291
bvn3-val1	2000 / 2348	966	1581	1940	5,643	21,845	333 / 290
bvn3-val2	2000 / 2348	969	1599	1894	5,547	20,835	340 / 295
bvn3-val3	1300 / 2048	692	1143	1301	12,169	23,276	333 / 291
bvn3-val4	1300 / 2048	685	1156	1294	12,587	24,014	336 / 290

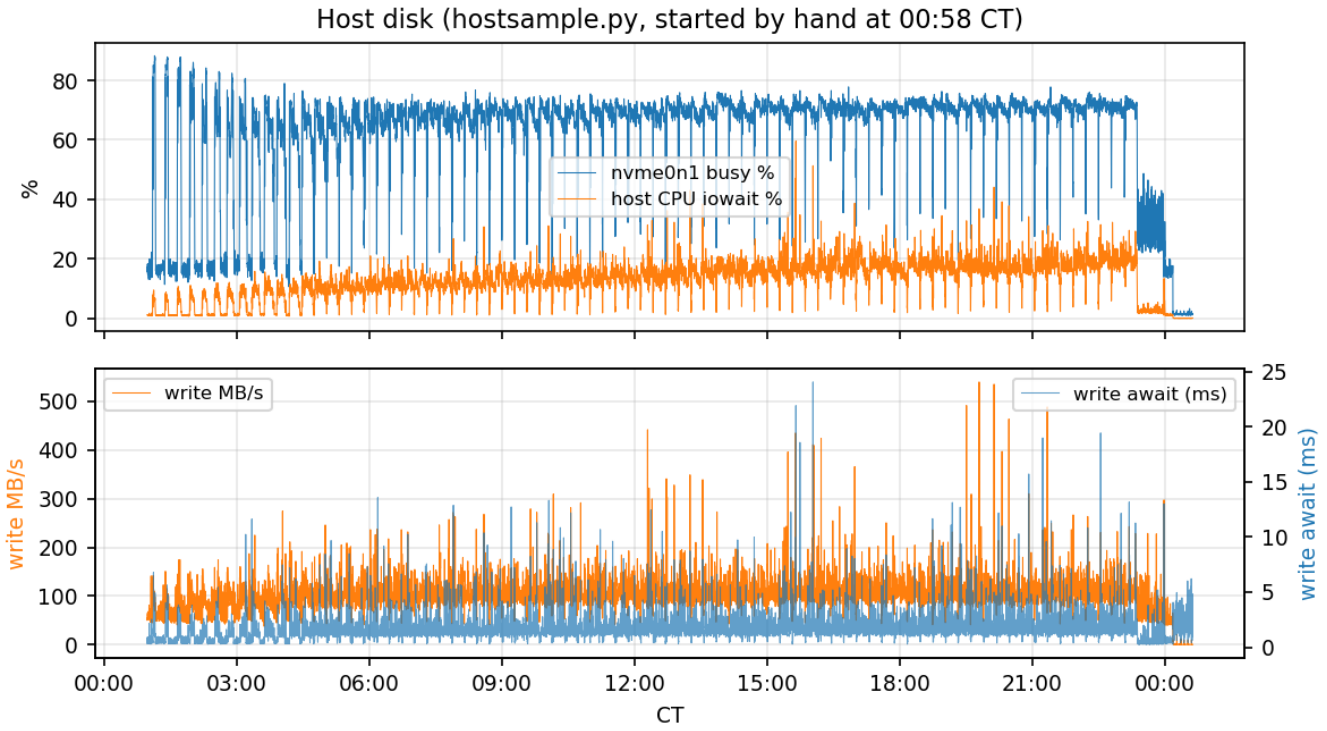
RSS in MiB. Reading:

- **The 2000 MiB arm** climbs ~85 MiB/h, then releases 300–400 MiB arm-wide twice (about 07:00 and 19:00 CT) — the pattern of a cache or retention window emptying, not a leak that only a restart would clear. Peaks 1.89–2.03 GiB, under the 2348 MiB container limit by ≥ 300 MiB. The BVN2 pair runs ~250 MiB hotter than BVN1/BVN3 all day.
- **The 1300 MiB arm** reaches its GOMEMLIMIT at about hour 13 and sits on it (≤ 1.33 GiB) for the rest of the day, paying 2.2× the GC cycles (11,700–12,600 vs 5,550–6,080) at roughly the same CPU. That is the GC holding the limit as designed, not thrashing: the executed heights of the two arms never separated and CPU per node differs by < 10 %.
- The RSS alarm in the monitor assumed 2048 MiB for every container, so the 2000 arm's "OOM approaching" lines above ~1.9 GiB were false — #4568.
- Goroutines were flat, 290–344 on every validator.

Reads

Read-back probe, validators, whole run: 426,034 timed reads, p50 1.7 ms, p95 13.9 ms, p99 59.5 ms, max 1,031.7 ms (one chain read on BVN2 of an entry 16,102 blocks old); 0 failed, 0 timed out. The follower that ran all day (`bvn1-fol1` while present, 32,644 reads of entries it holds): p50 0.3 ms, p95 1.8 ms, 289 failed — those are reads addressed to it in the seconds around its removal. The load generator had 15,537 NotReady answers retried at another endpoint (a joining follower's designed answer) and 0 queries no endpoint would answer.

Host



`hostsample.py` was started by hand an hour in (#4569: `soak.sh` does not start it), so the first hour has no disk samples.

	median	max
nvme0n1 busy	69 %	88 %
write	99 MB/s	—
write await	1.5 ms	24 ms
host CPU iowait	13 %	60 %
host CPU idle	65 %	—

The store grew at roughly 7.8 GB/h across the project's volume (read by hand from `docker system df` during the run: ~173 GB at hour 22; the harness does not record volume size). The disk was never the limiter: the 24 ms worst write wait is one sample, and the busy figure's periodic dips are the follower removals.

What this implies for a network running for years

A day is 86,000 blocks and 450,000 transactions. A year is 31 million blocks per partition and, at 5 tps, 158 million transactions. Every quantity below was measured over one day; what matters for years is its *slope* across the day, and whether the slope is tied to time, to block count, or to the size of the state. Where the run cannot separate those, it says so.

What was flat

- **Block production**: 0.99 blocks/s from block 40 to block 85,674. Nothing in commit or consensus slowed with the block count in this range.
- **Goroutines**: 290–344 on every validator all day — no goroutine leak at this scale.

- **Follower agreement**: the all-day follower matched the validators' root and BPT on 163,045 blocks with 0 mismatches; the lag never exceeded 1 block.
- **Delivery**: 0 heals, 0 delivery stalls, 0 reconcile pulls. Nothing accumulated in the synthetic or anchor streams.

What grew, and with what

quantity	hour 1–3	hour 20–22	slope	tied to
added → ACTIVE (join)	41 s	77 s	+4 ms per account in the universe	state size (accounts), not block count
live heap floor, 2000 arm (min heapAlloc per hour)	328 MiB	446 MiB	+11 MiB/h between releases; ~+2.5 MiB/h net of them	unknown: state size or time
live heap floor, 1300 arm	323 MiB	436 MiB	same	same
CPU per validator per hour	790 s (2000) / 814 s (1300)	1,013 s / 1,074 s	+28–32 % over the day at constant 5 tps	unknown: state size or uptime
read latency by entry age, p50 / p95	1.2 / 2.3 ms (< 5,000 blocks old)	2.4 / 16.6 ms (> 20,000 blocks old)	roughly doubles across the day's range	entry age in blocks
store, whole project	—	~173 GB at hour 22	~7.8 GB/h ≈ 0.5 GB/h per node	transactions, not time

Joins

The join pulls state and heads, not history (the design), so the chain's age does not enter it: 85,674 blocks of history cost the last joins nothing the first joins did not pay. What it pays for is accounts — about 4 ms each, measured across a universe growing 131 → 20,305. Extrapolated linearly, one follower stays inside the one-hour bar up to roughly 900,000 accounts; beyond that the state pull itself exceeds the bar, and the serial records phase (#4552, open for a decision) is the item that changes the slope. A years-old network also has years of history for the background fill; this run had at most 24 hours of it, and the background fill of real history was not measured.

Memory

The live heap — the floor the GC gets back to, read as the hour's minimum `heapAlloc` — rose 11 MiB/h on both arms, identically, from 328 MiB to 529 MiB over 18 hours; it was released twice (hours 7 and 19), and each release landed about 30 MiB above the previous one: 328 → 358 → 388 MiB. Net of the releases the floor climbs ~2.5 MiB/h, 60 MiB/day.

The hourly heap profiles name what the 19:00 CT release was, on `bvn2-val1` (18:06 → 19:07 CT, in-use heap 598 → 366 MB): anchor envelopes decoded for block production (`BlockAnchor` via `adapter.blockEnvelopes`, held somewhere after `ProduceBlock`) 88.5 → 6.5 MB, the BlockchainDB `immutableCache` 61 → 36 MB, and the receipts and Merkle states inside those envelopes. What triggers the release — a retention window of about 12 hours, or a store event — was not identified (filed as a question, #4571).

For years the question is whether the ~60 MiB/day net climb is **retained state** (a cache keyed by accounts, which would plateau when the account set stops growing — and this run's account set grew 155× in a day, so anything proportional to it looks exactly like a leak) or **retained time** (something held per block or per anchor that no release empties). If it were linear and unbounded, the 1300 MiB arm's live heap would reach its limit in roughly two weeks (≈ 800 MiB of headroom at 2.5 MiB/h), after which the GC thrashes and then the container is killed. The run cannot distinguish the two; a run with a fixed universe can (below).

CPU

Per-validator CPU per hour rose about 30 % across the day at a constant 5 tps and a constant 1 block/s. The same confound applies: a larger state makes every transaction cost more (BPT depth, cache misses), and separately the validators are known to slow with uptime in ways a restart resets. Over years neither can be left as it is: a 30 %/day rise at constant load is not sustainable even if it flattens, and the run did not see it flatten.

Reads

Reads of an entry 20,000+ blocks old cost twice what reads under 5,000 blocks cost at p50, and seven times at p95. In a years-old network almost every read is of an old entry — a year is 31 million blocks — and this run measured the latency only out to ~85,000 blocks of age. Whether the curve is logarithmic (a tree walk) or worse (a scan of history pages) is the thing to measure; the run does not say.

Storage

The store grows with blocks as much as with transactions (see "On separate servers for a year": about half the record writes are per block × validators), and nothing in it is reclaimed. At 5 tps the whole 16-node project wrote ~7.8 GB/h — about 0.5 GB per node-hour, or **~4 TB per node-year at 5 tps**, scaling with the rate. The node logs are a second stream of the same order (8.6 GB across 16 nodes in 24 h, ~0.2 TB per node-year) and are a deployment matter — rotate them.

The run that answers the years question

This run could not separate "grows with the state" from "grows with time", because the load generator grew the state all day. The run that can:

1. **Fixed universe**: the load generator stops adding accounts after, say, 10,000 and keeps transacting among them at 5 tps for 72 h or more. If the heap floor, CPU/h and join time flatten when the universe does, they are state-bound and the year's cost is set by the account count. If they keep climbing, they are time-bound and are defects.
2. **One restart at hour 48** on half the validators: whatever drops at the restart and climbs again is uptime, not state.
3. **Reads binned to 250,000 blocks of age** (the probe's reservoir needs to keep older entries), to get the shape of the latency curve.
4. **A join against an aged store**: start a follower against the 72-hour network and time the background history fill, which this run never exercised.

Until that run, the honest statement is: a day of the follower walk shows no cost tied to block count, a join cost tied to the account count at 4 ms per account, and three climbs — live heap, CPU per hour, and old-entry read latency — whose cause the day cannot name.

On separate servers for a year

The deployment to evaluate: every node on its own server — 8 cores, one 1 TB SSD, a high-speed internet link — for a year, at this run's load (5 tps) and shape (3 BVNs × 4 validators, Directory on all 12, 1 s blocks). The run put 17 containers on one 24-core host with one shared NVMe and a 66 GB page cache, ~0.05 ms between nodes. Below, each resource is taken per node from the run, then carried to a year; where the run is pessimistic or optimistic about a dedicated server, it says which.

resource	per node, measured	per node, a year at 5 tps	on 8 cores / 1 TB / fast link
CPU	median 0.25 cores, p95 0.5–0.65, peak 1.05 (validators); 0.57 median while joining	+30 %/day climb, cause unresolved	3 % of the box; fine unless the climb is unbounded
memory	2.03 GiB peak (2000 MiB arm), 1.33 GiB (1300 arm)	live heap +60 MiB/day net (#4571)	2–4 GiB container; the climb is the risk, not the level
store	~13.5 GB/day (~0.56 GB/h)	**~4.9 TB**; ~2 TB at zero load	**1 TB is full in ~75 days; ~5 months idle**
SSD writes	~6 MB/s (46× the growth)	~190 TB written	consumer 1 TB (≈600 TBW) ≈ 3 years of endurance
SSD reads	~3.7 MB/s, ~630 reads/s	—	trivial; depends on the page cache
network	not measured (#4579); ≥ 1.3 Mbit/s received by what is persisted	≥ 0.4 TB/month	bandwidth irrelevant; latency adds 20–40 % to a join

Storage is the binding constraint

The store grew ~7.8 GB/h across ~14 node stores: 13.5 GB per node-day, 4.9 TB per node-year at 5 tps, and nothing is reclaimed. A 1 TB SSD holds about 75 days.

Most of that is not user transactions. The BlockchainDB shape counters (`storage-stats/*.json`) show a BVN database writing 27 `Message.Main` records per block and the Directory database 38, against 1.8 user transactions per BVN block. Each Directory → BVN anchor arrives with the 12 Directory validators' signatures (13 of the 27); the Directory records each BVN anchor with its 4 validators' signatures plus its own (most of the 38). Roughly **half of all record writes are per block × validators, independent of load**. Every node holds both its BVN's database and the Directory's, so the per-node figure already contains both. Because the Directory's validators are the BVN validators, both terms scale with the total validator count N (≈ N + 3 messages per Directory block, N + 1 per BVN block, ≈ 2N per second per node), not with the number of BVNs as such; a fourth BVN of 4 validators adds 5 messages per block to every node's Directory database and 4 signatures to every BVN's anchor. At 1 s blocks and N = 12 that is a floor on the order of 2 TB per node-year with no users at all (≈ 2.5× at N = 30) — counts, not bytes, because the harness does not record bytes per shape, so the proportion is approximate. Filed as #4580: the choices are blocks only when there is something to commit (quiescence was given up from Kourou, #4277), not storing every validator's signature once an anchor is certified, history offload, or all three.

Write amplification is the second storage finding: the host wrote 101 MB/s to grow the store 2.2 MB/s, 46×. Per node that is ~6 MB/s, 190 TB a year — within an enterprise SSD's rating, about three years of a consumer drive's. Reads were 3.7 MB/s and 630/s per node, nothing for any SSD; but the run's host had 66 GB of page cache over a 173 GB store, and the old-entry read latency (p95 16.6 ms at > 20,000 blocks) was measured with that cache. A server with 16 GB of RAM in front of a 1 TB store will see those reads go to the SSD; 32 GB is the sensible minimum.

On a dedicated SSD the run's disk pressure disappears: the shared NVMe at 69 % busy becomes ~4 % per node, and the Monday fstrim stall that killed an earlier run (#4559's history) cannot recur. The run is pessimistic about disk *contention* and says nothing about disk *capacity* except that it will run out.

CPU and memory are not — unless the climbs are real

A validator used a quarter of a core at the median and one core at peak; a follower joining used 0.57 cores while it pulled state. Eight cores are 3 % used. The climb — CPU per hour +30 % across the

day at constant load — is the only thing that could change that: taken linearly and unbounded it fills 8 cores in about four months, but the run cannot say whether it is bound to the account set (which grew 155× in the day) or to uptime. The same holds for memory: 2 GiB peak is nothing on a server, and a 4 GiB GOMEMLIMIT halves the GC work seen here; the live heap's +60 MiB/day net climb (#4571) would reach a 4 GiB limit in about two months if unbounded. Both are settled by the fixed-universe run in the previous section, not by hardware.

A year also contains restarts — upgrades, reboots — and this run had none. Restart behaviour is proven on the restart branch (#4502, #4545), not here.

The network link

Bandwidth was not recorded (#4579). A lower bound is what a validator persists, 13.5 GB/day ≈ 1.3 Mbit/s received; with gossip fan-out and the headers, votes and certificates of two partitions at 1 block/s, a few Mbit/s is the likely figure — under 1 % of a gigabit link, but 0.5–2 TB a month against a metered VPS allowance.

Latency is where the run is optimistic. Docker gave 0.05 ms between nodes; the internet gives 20–100 ms. Consequences:

- **Consensus**: a DAG round is at least two message hops; at 100 ms RTT that is inside the 1 s block, but a Directory of 12 validators across continents is RTT-bound in a way this run did not exercise.
- **Joins**: the records phase pulls 64 states per call (`join/state.go`, `statesChunk`). The 4 ms per account measured is ~0.25 s of validator-side work per call, so a 20,000-account join is ~320 calls; adding 50–100 ms RTT to each is +16–32 s on a 77 s join (+20–40 %). The one-hour bar is then reached near 700,000 accounts rather than 900,000.
- **Follower lag**: ≤ 1 block in the run, bound by gossip propagation; at 100 ms RTT still sub-second.

The year, in one paragraph

On 8 cores, 1 TB and a fast link, at 5 tps, a node from this run runs out of disk in about 75 days and would run out in about 5 months with no load at all; that is the finding that decides the deployment, and it is a protocol choice (#4580), not a hardware one. CPU and memory are each under 5 % of the box, and the link is under 1 % used; the three climbs the day recorded — CPU per hour, live heap, old-entry read latency — are the only way those change, and whether they stop when the account set stops growing is the one question the next run must answer. Internet latency makes joins 20–40 % slower and consensus RTT-bound, neither of which this run measured. Logs need rotation (~0.2 TB per node-year). Restarts are not covered.

Things to know about the record

- `manifest.md` carries a heading "Stopped early by stallkill … monitor unreachable for 120 s" dated 05:00:00Z. The driver had already exited cleanly (exit 0, 24.01 h) at 04:58:03Z; the watchdog then found the monitor gone and tore the project down as it is meant to. The heading is wrong for a run that finished, and `soak.sh` reads it as `stopped_early=stallkill` — filed as a harness defect.
- `seizewatch` printed one SEIZED line at 00:11 CT (16 min in): a synthetic stream BVN2 → Directory with 8 undelivered for 20 polls. Nothing else saw it: 0 delivery-stall captures, 0 stalled channels at the end, 0 heals, and the baseline runs carried the same line. Treated as a watchdog artefact, not a network event.
- The manifest's "follower start rejoined … not established" rows for the three late followers describe their silence after their last removal (the walk removes the container; it is silent by design).
- `headers/votes dropped by validators for a non-committee author` are not measured: those are Debug lines without a module attribute, so this build never emits them (noted in the manifest).
- Raw data (13 GB, mostly `node-logs-live.txt`) lives in `test/docker/soak/runs/20261013T045549Z/` on thelio and is not committed; `manifest.md`, `follower-report.md`, this report and the conf are.

Follow-ups filed

- **#4568** — the monitor's RSS alarm must read each container's `HostConfig.Memory`; the manifest should report both arms.
- **#4569** — `soak.sh` should start `hostsample.py` itself.
- **#4570** — manifest "Stopped early" heading after a clean finish.
- **#4579** — the hourly `docker stats` capture drops NET I/O and BLOCK I/O; the store size is read by hand.
- **#4580** — ~half the store's record writes are per block × validators, independent of load; a 1 TB node fills in months.
- **#4571** — what holds ~90 MB of decoded anchor envelopes for ~12 h on every validator, what releases them, and whether the ~60 MiB/day residual is per account or per block..
- #4559 left open for the decision on 5 vs 10 tps.

Appendix: every join

Added time in CT; “filled” = every section loaded, minutes after the add (`joinphases.py`); worst behind = the largest lag in blocks seen during the 900 s hold; polls at 10 s.

#	added (CT)	follower	universe	→ ACTIVE s	filled min	worst behind BVN / DN	polls not ACTIVE	verdict
1	Oct 12 23:59	bvn1-fol1	131	21	0.2	0 / 0	0 of 90	PASS
2	Oct 13 00:15	bvn2-fol1	450	41	0.4	1 / 0	0 of 90	PASS
3	Oct 13 00:32	bvn3-fol2	765	41	0.4	1 / 0	0 of 90	PASS
4	Oct 13 00:49	bvn1-fol1	1,111	41	0.4	1 / 0	0 of 90	PASS
5	Oct 13 01:05	bvn2-fol1	1,426	41	0.4	0 / 0	0 of 90	PASS
6	Oct 13 01:22	bvn3-fol2	1,742	41	0.4	1 / 0	0 of 90	PASS
7	Oct 13 01:39	bvn1-fol1	2,080	41	0.4	1 / 0	0 of 90	PASS
8	Oct 13 01:56	bvn2-fol1	2,372	41	0.5	1 / 0	0 of 90	PASS
9	Oct 13 02:13	bvn3-fol2	2,677	51	0.5	1 / 0	0 of 90	PASS
10	Oct 13 02:30	bvn1-fol1	2,991	51	0.5	1 / 0	0 of 90	PASS
11	Oct 13 02:46	bvn2-fol1	3,304	51	0.5	1 / 0	0 of 90	PASS
12	Oct 13 03:03	bvn3-fol2	3,585	51	0.5	1 / 0	0 of 90	PASS
13	Oct 13 03:20	bvn1-fol1	3,863	41	0.5	0 / 0	0 of 90	PASS
14	Oct 13 03:37	bvn2-fol1	4,160	51	0.5	1 / 0	0 of 90	PASS
15	Oct 13 03:54	bvn3-fol2	4,444	51	0.5	1 / 0	0 of 90	PASS
16	Oct 13 04:11	bvn1-fol1	4,763	51	0.6	1 / 0	0 of 90	PASS
17	Oct 13 04:28	bvn2-fol1	5,081	61	0.6	1 / 0	0 of 90	PASS
18	Oct 13 04:45	bvn3-fol2	5,345	51	0.6	1 / 0	0 of 90	PASS
19	Oct 13 05:02	bvn1-fol1	5,675	51	0.5	1 / 0	0 of 90	PASS
20	Oct 13 05:19	bvn2-fol1	5,968	51	0.6	1 / 0	0 of 90	PASS
21	Oct 13 05:36	bvn3-fol2	6,272	51	0.6	1 / 1	0 of 90	PASS
22	Oct 13 05:53	bvn1-fol1	6,564	51	0.6	1 / 0	0 of 90	PASS
23	Oct 13 06:10	bvn2-fol1	6,874	51	0.6	1 / 2	0 of 90	PASS
24	Oct 13 06:27	bvn3-fol2	7,139	51	0.6	1 / 0	0 of 90	PASS

#	added (CT)	follower	universe	→ ACTIVE s	filled min	worst behind BVN / DN	polls not ACTIVE	verdict
25	Oct 13 06:44	bvn1-fol1	7,435	52	0.6	1 / 0	0 of 90	PASS
26	Oct 13 07:01	bvn2-fol1	7,719	51	0.6	1 / 0	0 of 90	PASS
27	Oct 13 07:18	bvn3-fol2	7,993	51	0.6	1 / 0	0 of 90	PASS
28	Oct 13 07:35	bvn1-fol1	8,299	51	0.6	1 / 0	0 of 90	PASS
29	Oct 13 07:52	bvn2-fol1	8,611	51	0.6	1 / 0	0 of 90	PASS
30	Oct 13 08:09	bvn3-fol2	8,924	51	0.7	1 / 0	0 of 90	PASS
31	Oct 13 08:26	bvn1-fol1	9,220	61	0.7	0 / 1	0 of 90	PASS
32	Oct 13 08:43	bvn2-fol1	9,523	61	0.7	1 / 0	0 of 90	PASS
33	Oct 13 09:00	bvn3-fol2	9,827	51	0.7	1 / 0	0 of 90	PASS
34	Oct 13 09:17	bvn1-fol1	10,061	51	0.7	1 / 0	0 of 90	PASS
35	Oct 13 09:34	bvn2-fol1	10,333	51	0.7	1 / 0	0 of 90	PASS
36	Oct 13 09:51	bvn3-fol2	10,618	51	0.7	1 / 0	0 of 90	PASS
37	Oct 13 10:08	bvn1-fol1	10,858	61	0.7	1 / 0	0 of 90	PASS
38	Oct 13 10:25	bvn2-fol1	11,050	51	0.8	1 / 0	0 of 90	PASS
39	Oct 13 10:42	bvn3-fol2	11,262	71	0.8	1 / 0	0 of 90	PASS
40	Oct 13 10:59	bvn1-fol1	11,428	61	0.8	1 / 0	0 of 90	PASS
41	Oct 13 11:16	bvn2-fol1	11,618	61	0.8	1 / 0	0 of 90	PASS
42	Oct 13 11:33	bvn3-fol2	11,853	71	0.9	1 / 0	0 of 90	PASS
43	Oct 13 11:51	bvn1-fol1	12,057	62	0.8	1 / 0	0 of 90	PASS
44	Oct 13 12:08	bvn2-fol1	12,262	51	0.8	1 / 0	0 of 90	PASS
45	Oct 13 12:25	bvn3-fol2	12,574	51	0.8	1 / 0	0 of 90	PASS
46	Oct 13 12:42	bvn1-fol1	12,792	72	0.9	1 / 0	0 of 90	PASS
47	Oct 13 12:59	bvn2-fol1	13,064	71	1.0	1 / 0	0 of 90	PASS
48	Oct 13 13:16	bvn3-fol2	13,355	61	0.9	1 / 0	0 of 90	PASS
49	Oct 13 13:34	bvn1-fol1	13,588	62	0.8	1 / 0	0 of 90	PASS
50	Oct 13 13:51	bvn2-fol1	13,835	71	1.0	1 / 0	0 of 90	PASS

#	added (CT)	follower	universe	→ ACTIVE s	filled min	worst behind BVN / DN	polls not ACTIVE	verdict
51	Oct 13 14:08	bvn3-fol2	14,026	61	0.9	1 / 0	0 of 90	PASS
52	Oct 13 14:25	bvn1-fol1	14,194	62	0.9	1 / 0	0 of 90	PASS
53	Oct 13 14:42	bvn2-fol1	14,406	71	1.0	1 / 0	0 of 90	PASS
54	Oct 13 15:00	bvn3-fol2	14,615	62	1.0	1 / 0	0 of 90	PASS
55	Oct 13 15:17	bvn1-fol1	14,853	71	1.0	1 / 0	0 of 90	PASS
56	Oct 13 15:34	bvn2-fol1	15,113	71	1.2	1 / 1	0 of 90	PASS
57	Oct 13 15:51	bvn3-fol2	15,309	71	1.0	1 / 0	0 of 90	PASS
58	Oct 13 16:09	bvn1-fol1	15,532	61	1.0	1 / 0	0 of 90	PASS
59	Oct 13 16:26	bvn2-fol1	15,788	82	1.3	1 / 0	0 of 90	PASS
60	Oct 13 16:43	bvn3-fol2	16,008	71	0.9	1 / 0	0 of 90	PASS
61	Oct 13 17:01	bvn1-fol1	16,193	71	1.0	0 / 0	0 of 90	PASS
62	Oct 13 17:18	bvn2-fol1	16,396	72	1.4	1 / 0	0 of 90	PASS
63	Oct 13 17:35	bvn3-fol2	16,547	72	1.0	1 / 0	0 of 90	PASS
64	Oct 13 17:52	bvn1-fol1	16,693	81	1.2	1 / 0	0 of 90	PASS
65	Oct 13 18:10	bvn2-fol1	16,901	71	1.1	0 / 0	0 of 90	PASS
66	Oct 13 18:27	bvn3-fol2	17,158	71	1.0	1 / 0	0 of 90	PASS
67	Oct 13 18:45	bvn1-fol1	17,334	51	1.0	1 / 0	0 of 90	PASS
68	Oct 13 19:02	bvn2-fol1	17,564	72	1.0	0 / 0	0 of 90	PASS
69	Oct 13 19:19	bvn3-fol2	17,833	61	1.1	1 / 0	0 of 90	PASS
70	Oct 13 19:36	bvn1-fol1	18,056	82	1.2	0 / 0	0 of 90	PASS
71	Oct 13 19:53	bvn2-fol1	18,306	82	1.2	1 / 0	0 of 90	PASS
72	Oct 13 20:11	bvn3-fol2	18,492	72	1.0	1 / 0	0 of 90	PASS
73	Oct 13 20:28	bvn1-fol1	18,636	61	1.1	0 / 0	0 of 90	PASS
74	Oct 13 20:45	bvn2-fol1	18,805	72	1.1	1 / 0	0 of 90	PASS
75	Oct 13 21:03	bvn3-fol2	19,013	72	1.0	1 / 0	0 of 90	PASS
76	Oct 13 21:20	bvn1-fol1	19,267	81	1.1	1 / 0	0 of 90	PASS

#	added (CT)	follower	universe	→ ACTIVE s	filled min	worst behind BVN / DN	polls not ACTIVE	verdict
77	Oct 13 21:37	bvn2-fol1	19,429	71	1.1	0 / 0	0 of 90	PASS
78	Oct 13 21:55	bvn3-fol2	19,572	72	1.1	1 / 0	0 of 90	PASS
79	Oct 13 22:12	bvn1-fol1	19,725	71	1.2	1 / 0	0 of 90	PASS
80	Oct 13 22:29	bvn2-fol1	19,953	81	1.2	1 / 0	0 of 90	PASS
81	Oct 13 22:47	bvn3-fol2	20,144	102	1.6	1 / 0	0 of 90	PASS
82	Oct 13 23:05	bvn1-fol1	20,305	82	1.2	1 / 0	0 of 90	PASS